

EECS 482

Introduction to operating systems

Deadlock prevention; address spaces

Peter Chen
University of Michigan

How to eliminate hold and wait?

- First step:
 - Move resource acquisition to beginning

```
phase 1          all needed
                ×
while (!done) {
    acquire some resource
    work
}
```

```
phase 2
    release all resources
```

How to eliminate hold and wait?

- Second step
 - Option A: Acquire all needed resources **atomically**

phase 1

```
acquire all needed resources atomic  
while (!done) {  
    work  
}
```

phase 2

```
release all resources
```

Atomic acquisition

`L.lock()`

`while right chopstick busy or left chopstick busy`

`cv.wait (L)`

`pick up right chopstick`

`pick up left chopstick`

`L.unlock()`

`<eat>`

`L.lock()`

`drop left chopstick`

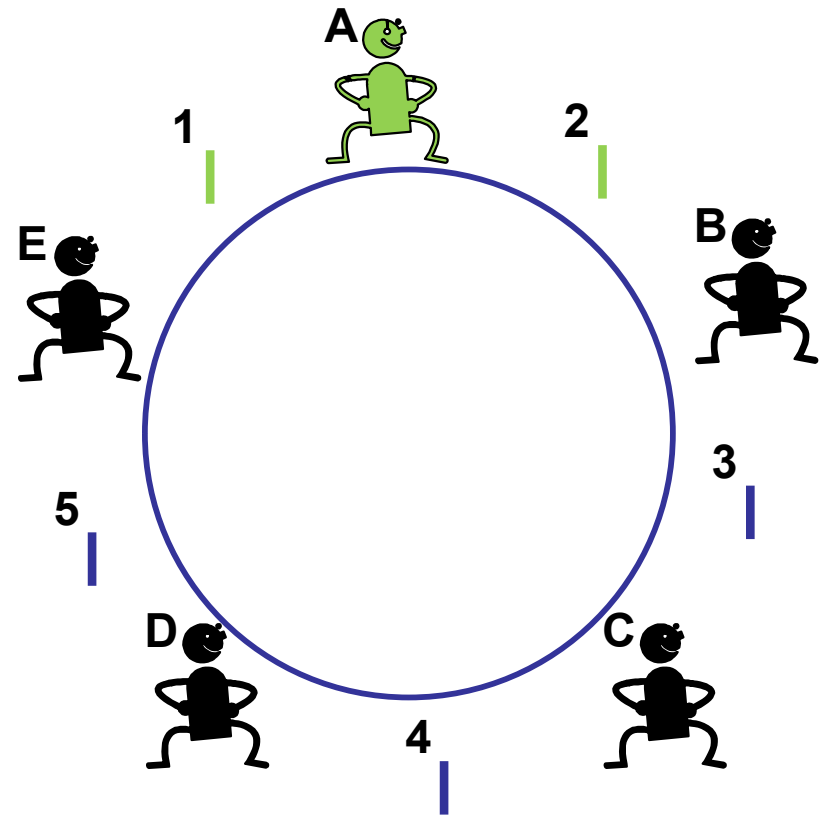
`drop right chopstick`

`cv.broadcast()`

`L.unlock()`

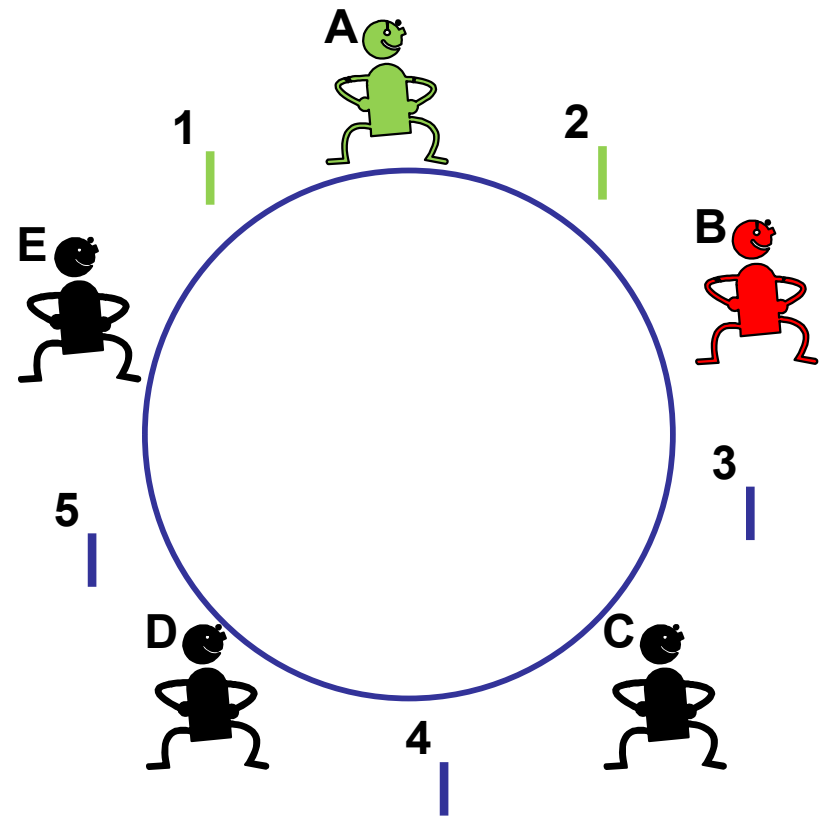
Dining philosophers

- A starts to eat



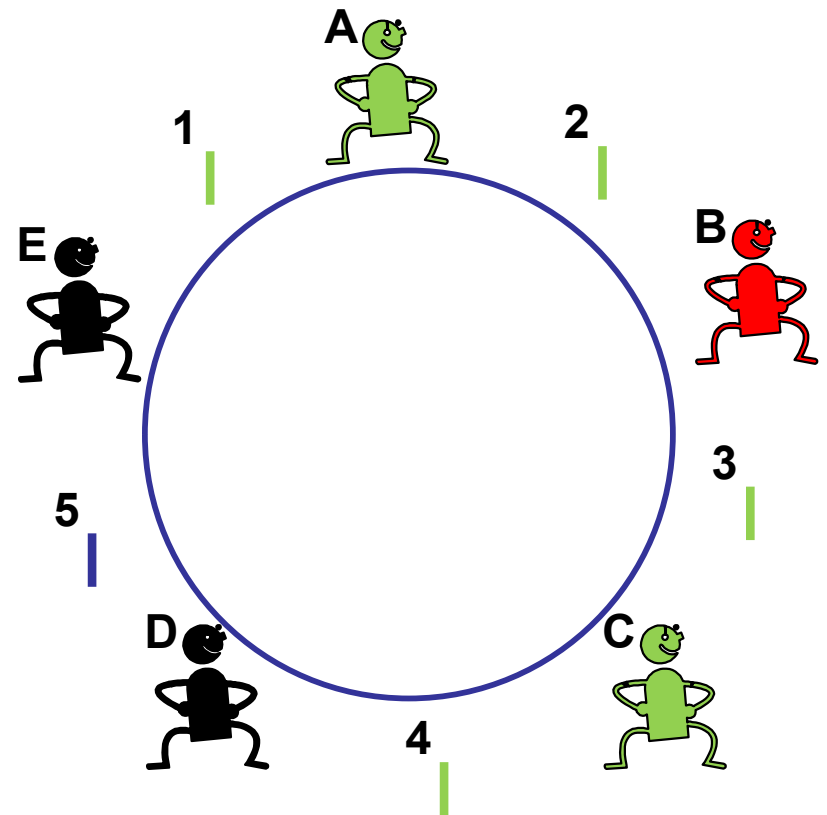
Dining philosophers

- A starts to eat
- B waits



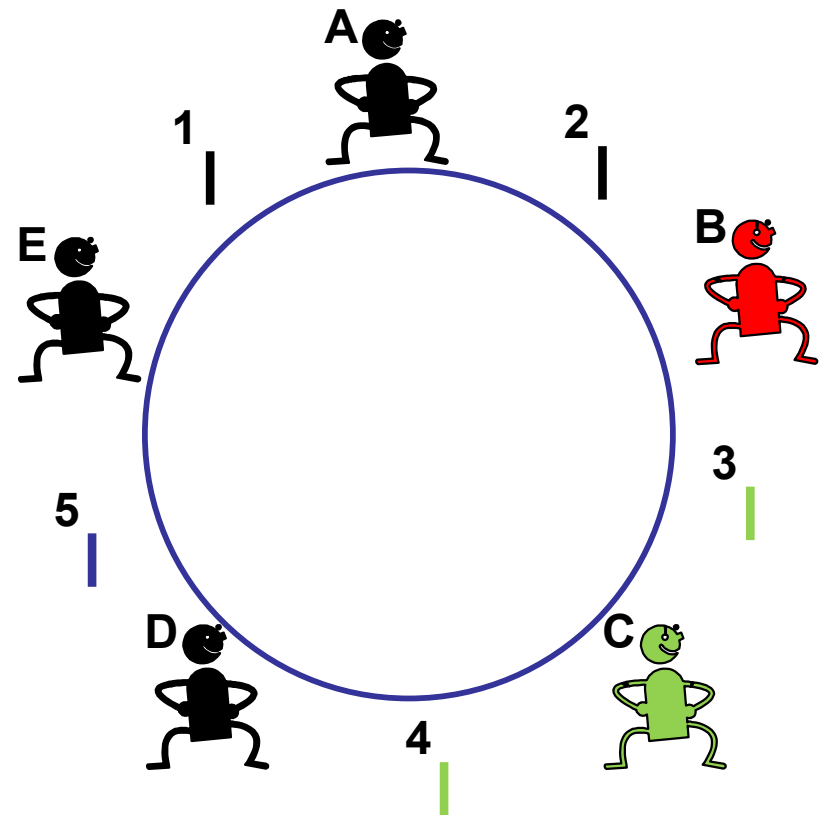
Dining philosophers

- A starts to eat
- B waits
- C starts to eat



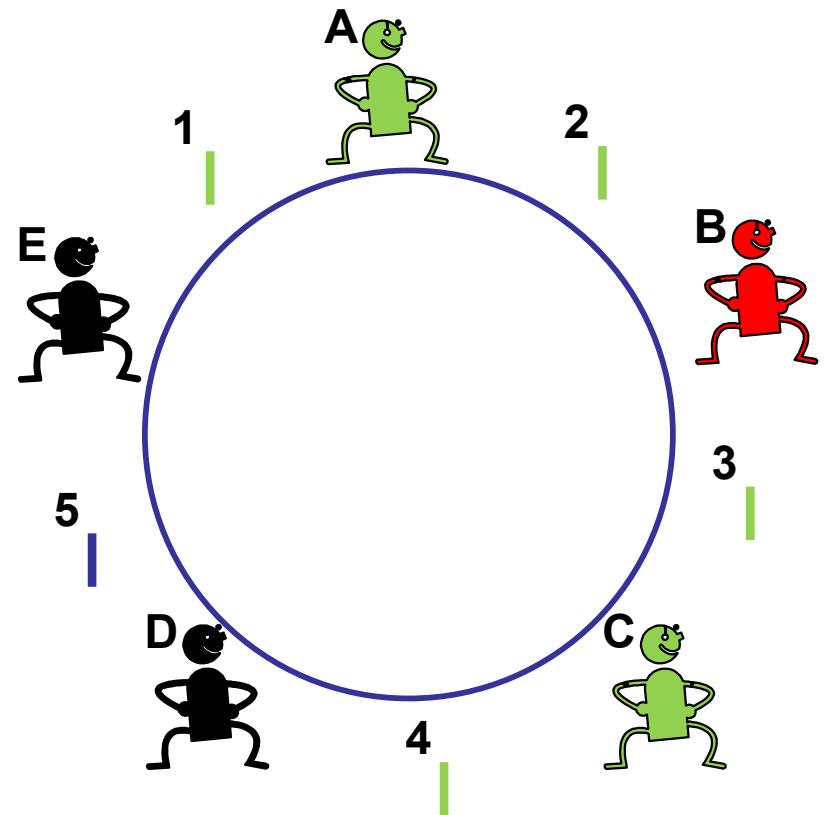
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes



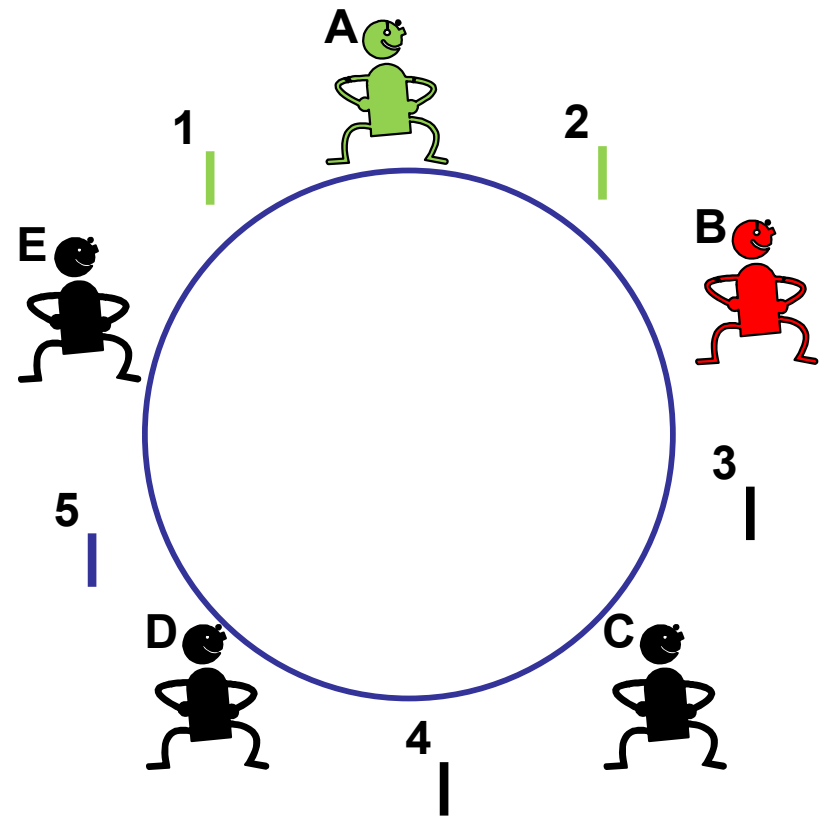
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat



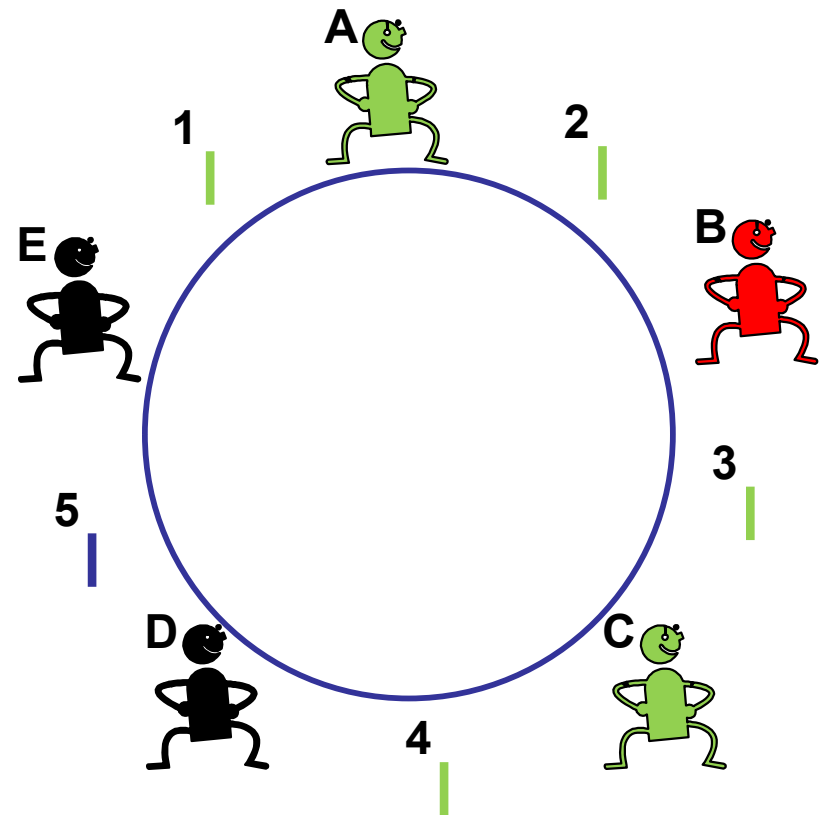
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes



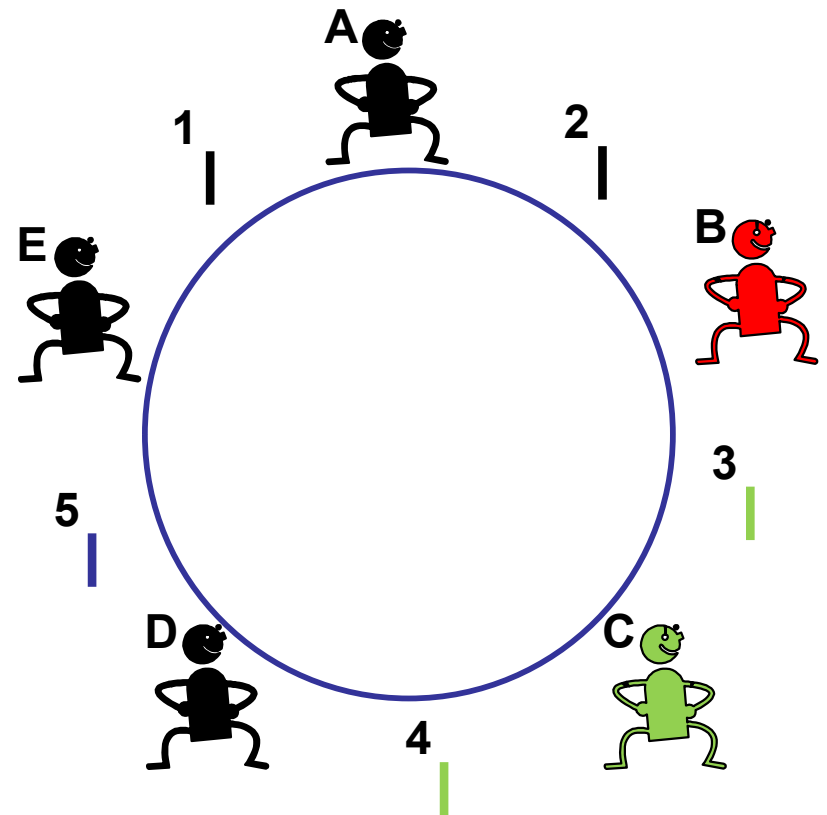
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat



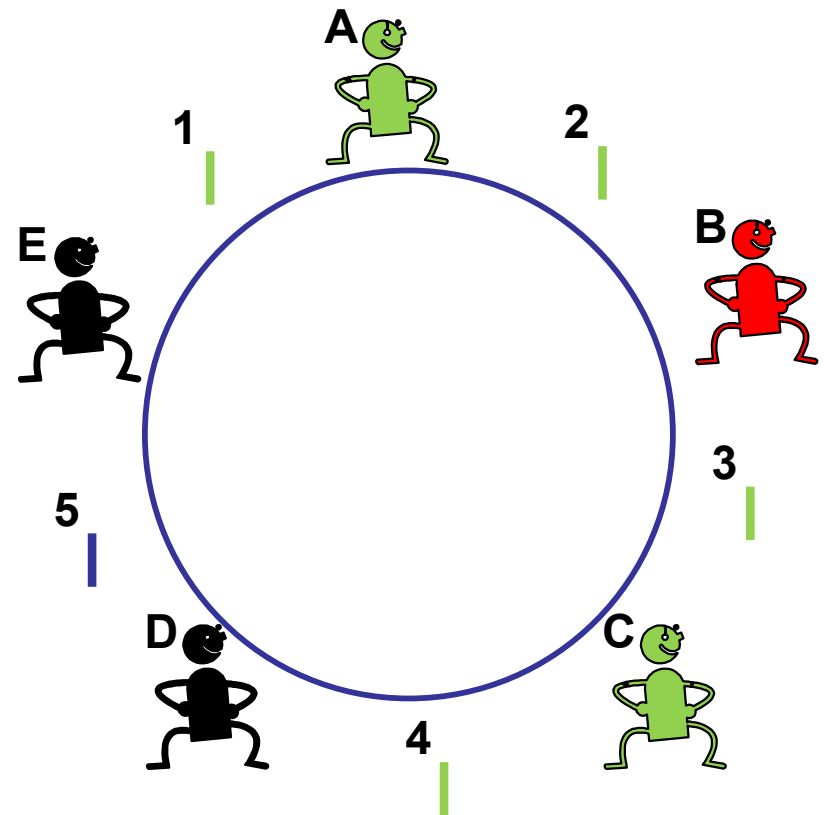
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat
- A finishes



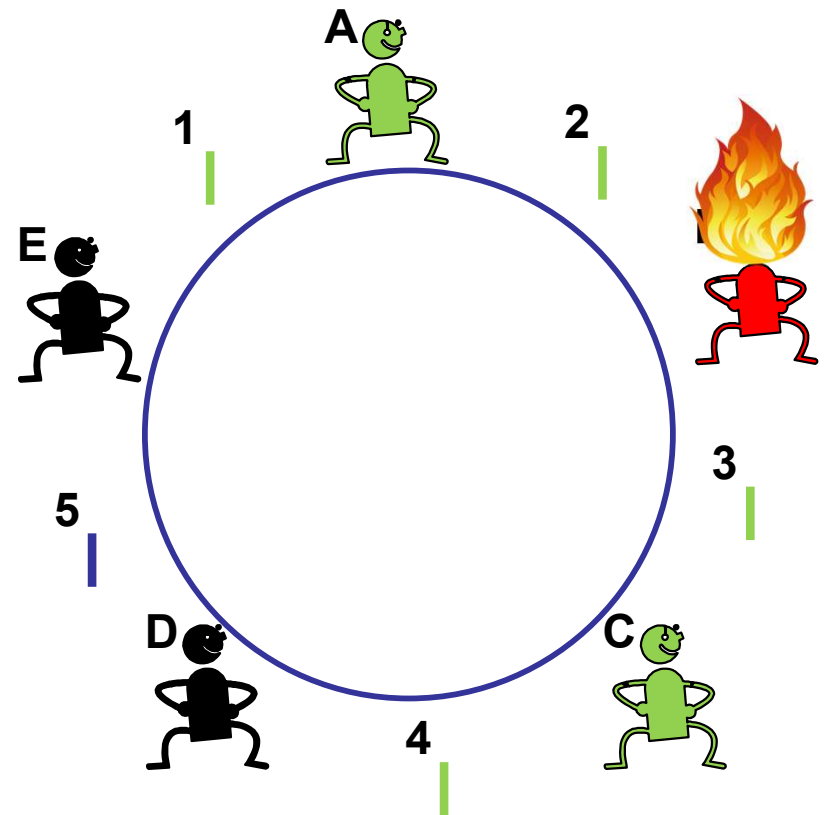
Dining philosophers

- A starts to eat
- B waits
- C starts to eat
- A finishes
- A starts to eat
- C finishes
- C starts to eat
- A finishes
- A starts to eat



Dining philosophers

- A starts to eat
- B waits
- while (1)
 - C starts to eat
 - A finishes
 - A starts to eat
 - C finishes



How to eliminate hold and wait?

- Second step
 - Option A: Acquire all needed resources **atomically**
 - Option B: Release and retry if you encounter a busy resource

phase 1

```
acquire all needed resources
while (!done) {
    work
}
```

phase 2

```
release all resources
```

How to eliminate hold and wait?

- What's the problem with this style of solution?

phase 1

```
acquire all needed resources
while (!done) {
    work
}
```

phase 2

```
release all resources
```


How to eliminate cycle of hold-and-wait?

- Define a global order over all resources
 - All threads **must** follow this order when acquiring resources
 - Guarantees that some thread can always make progress

Thread A

x.lock()

y.lock()

...

y.unlock()

x.unlock()

Thread B

y.lock()

x.lock()

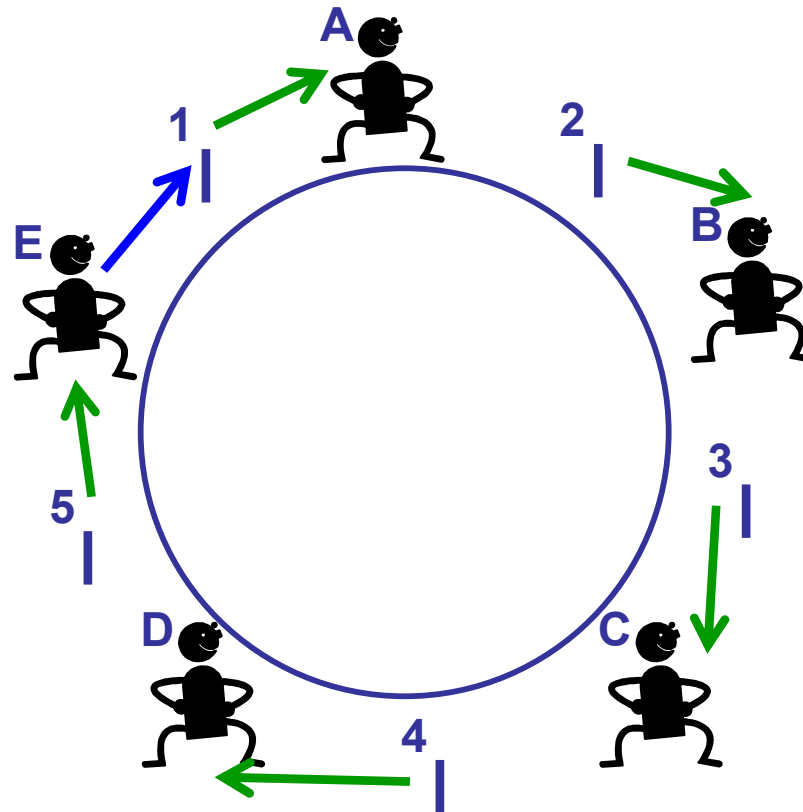
...

x.unlock()

y.unlock()

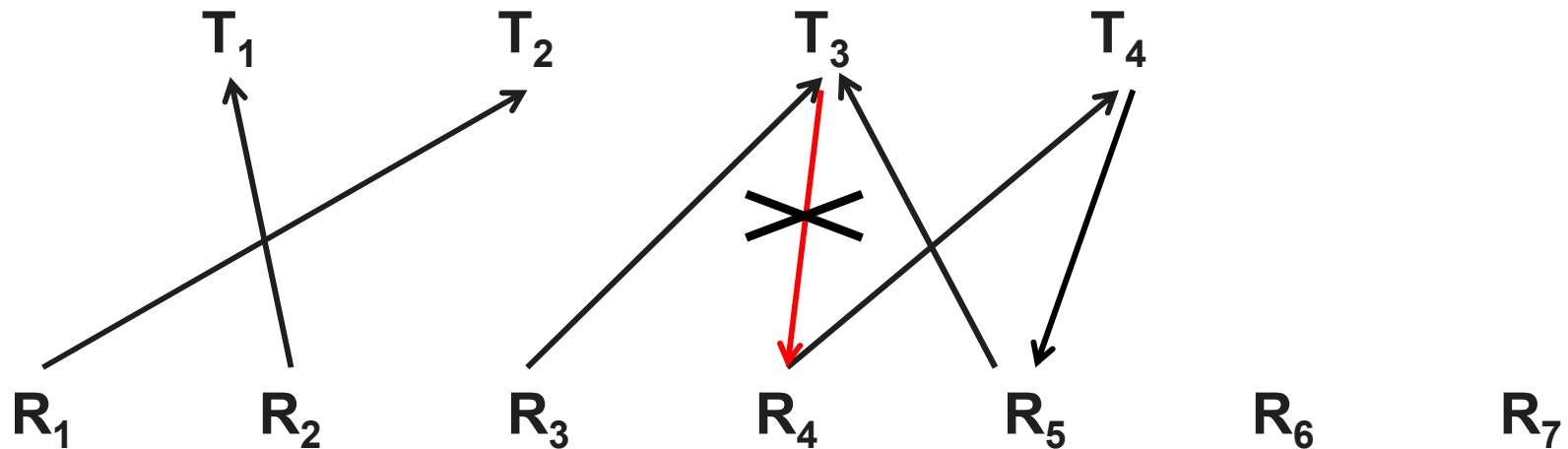
Dining philosophers

- Pick up lower # chopstick first
- Pick up higher # chopstick second



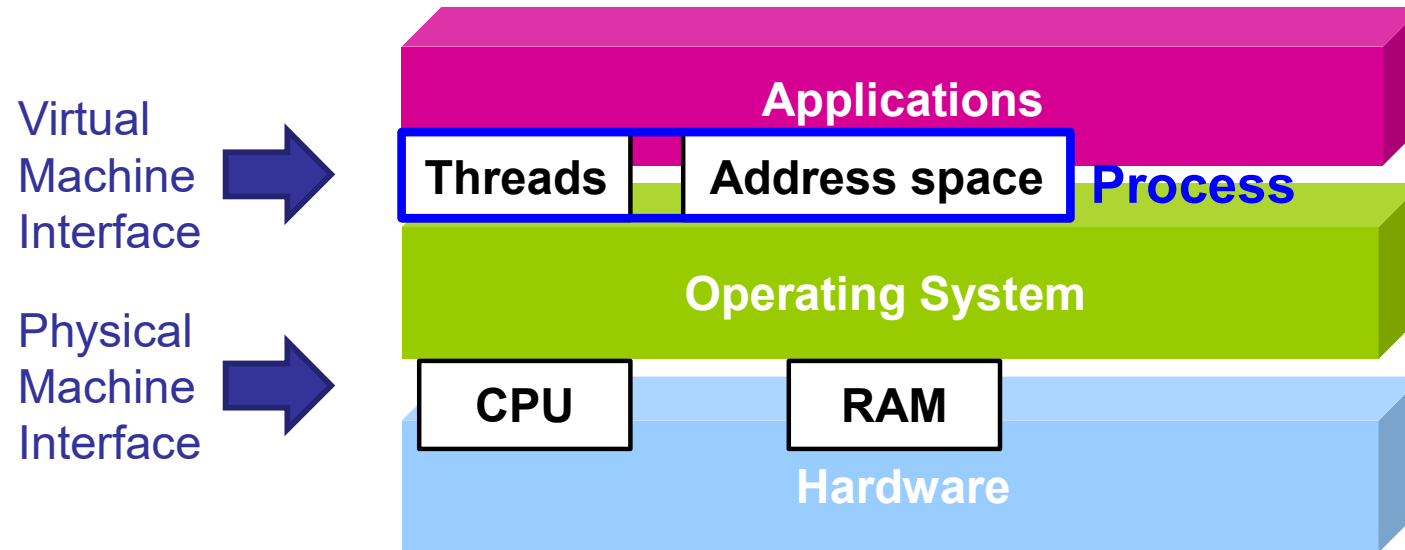
Global ordering of resources

- How can we be sure that *some* thread can make progress?



- Which thread can make progress?

OS abstractions



- Process = one or more threads in an address space

Address space

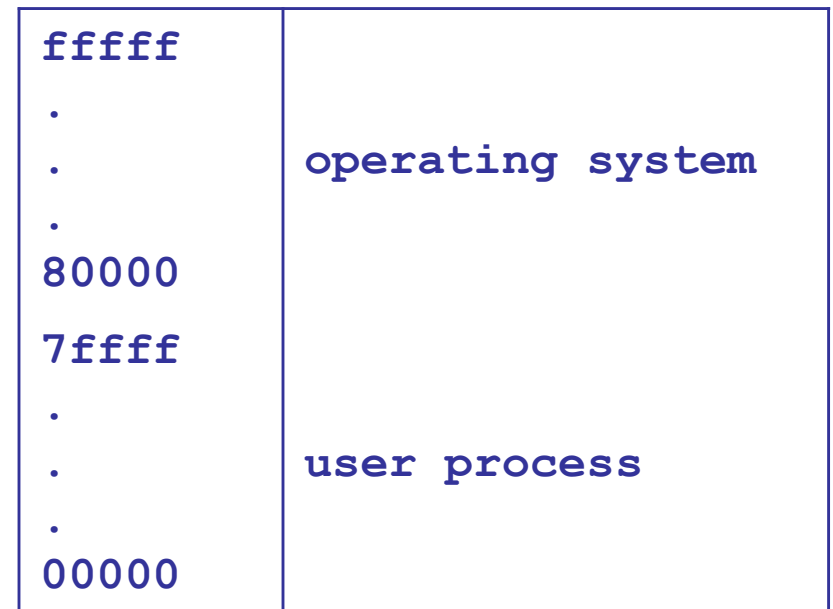
- All the memory space the process can use as it runs
 - Code
 - Stack
 - Data segment
- Physical machine interface: single memory shared by all jobs
- Virtual machine interface: each process has its own memory

Address space abstractions

- **Address independence**: same numeric address can be used in different address spaces, yet refer to distinct data items
- **Protection (controlled sharing)**: one process can't access data in another process's address space
- **Large address space**: address space for a process can be larger than physical memory

Uni-programming

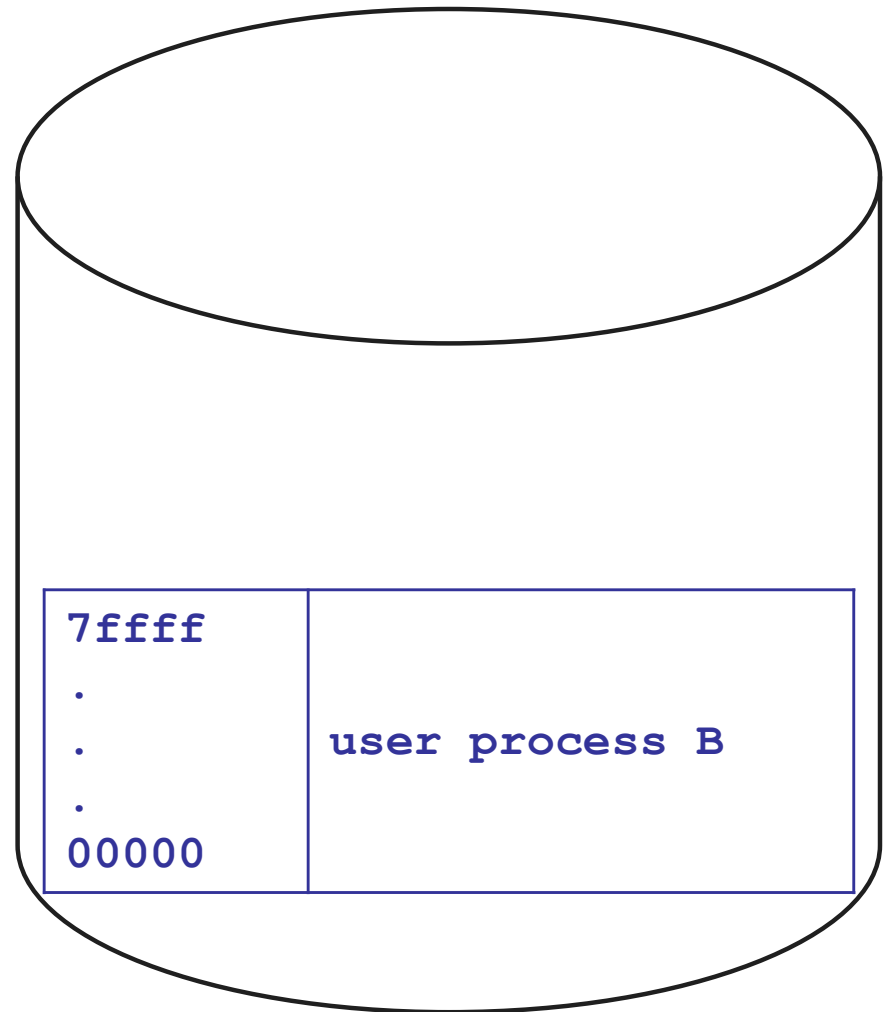
- 1 process occupies memory at a time
- Always load process into same spot in memory
- Reserve space for OS
- Virtual address (process view) = physical address (hardware view)



How to switch processes with uni-programming?

ffffff . . . 80000	operating system
7ffff . . . 00000	user process A

Physical memory



Disk

Uni-programming summary

- Address space abstractions
 - ✓ Address independence?
 - ✓ Protection?
 - ✗ Large address space?
- + Simple (early operating systems used this)
- Expensive context switch

Multi-programming

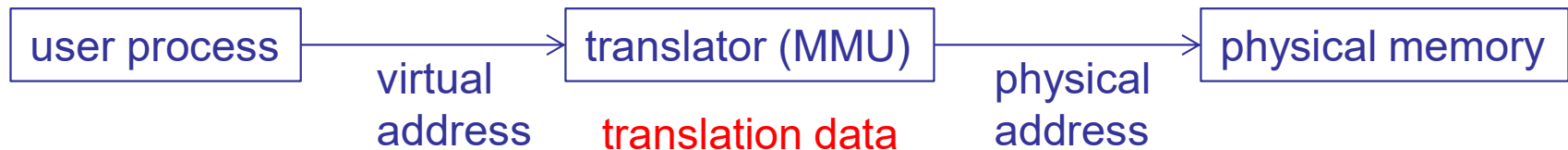
- Allow **multiple** processes to reside in physical memory at the same time
 - Makes it harder to provide protection
 - Makes it harder to provide address independence
- Providing protection and address independence requires system to do some work on **each memory access (!)**

Dynamic address translation



- **Address independence**
 - MMU translates same virtual address in different processes to different physical addresses
- **Protection**
 - MMU ensures that processes don't access each other's data
- **Large address space**
 - Virtual address only needs to be in physical memory when accessed

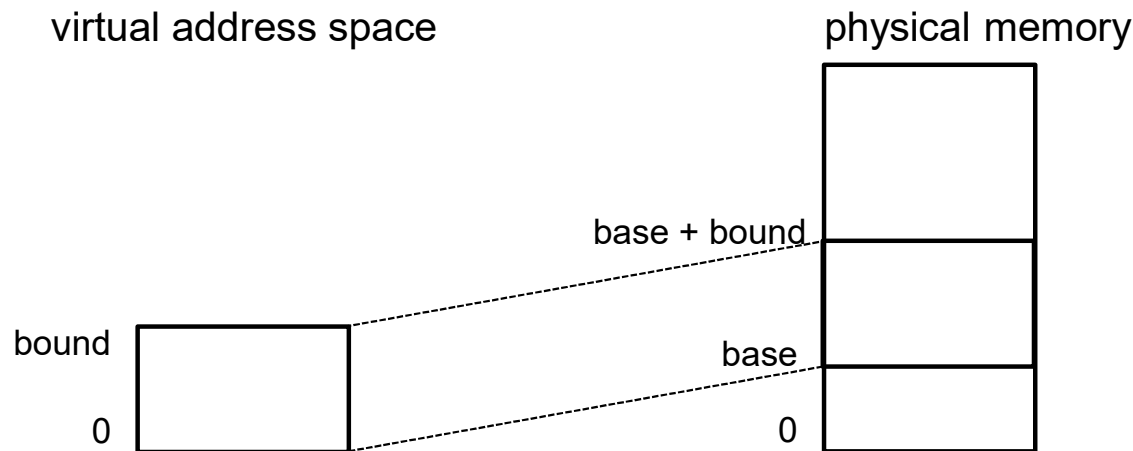
Dynamic address translation



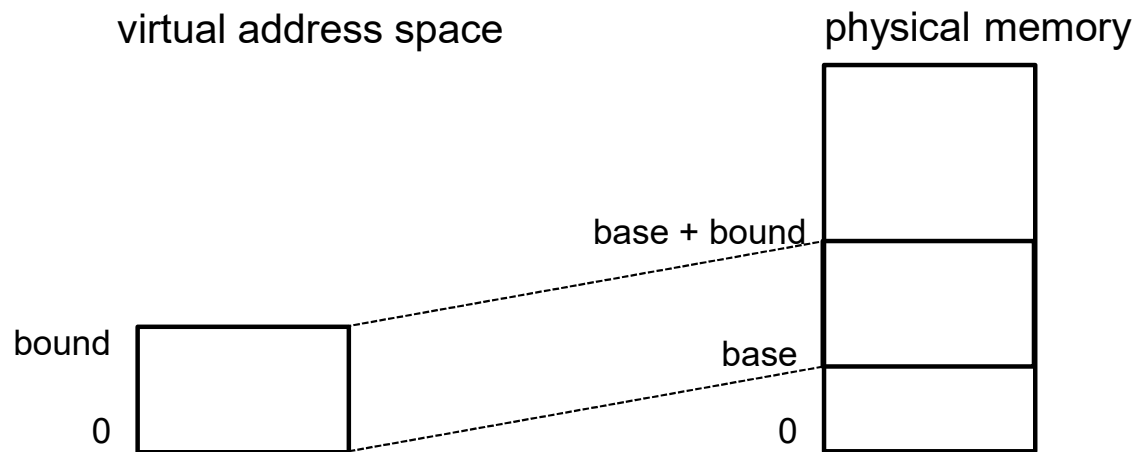
- Translator typically implemented as part of CPU
- Translator uses translation data
 - Each process has its own translation data
 - Changing address spaces = changing data used to translate a virtual address
- Many ways (data structures) to implement translator
 - **Speed** of translation
 - **Size** of data needed to support translation
 - **Flexibility** (sharing, growth, large address space)

Base and bound

- Load each process into a **single contiguous** region of physical memory
 - **Base** register: starting physical address
 - **Bound** register: size of region



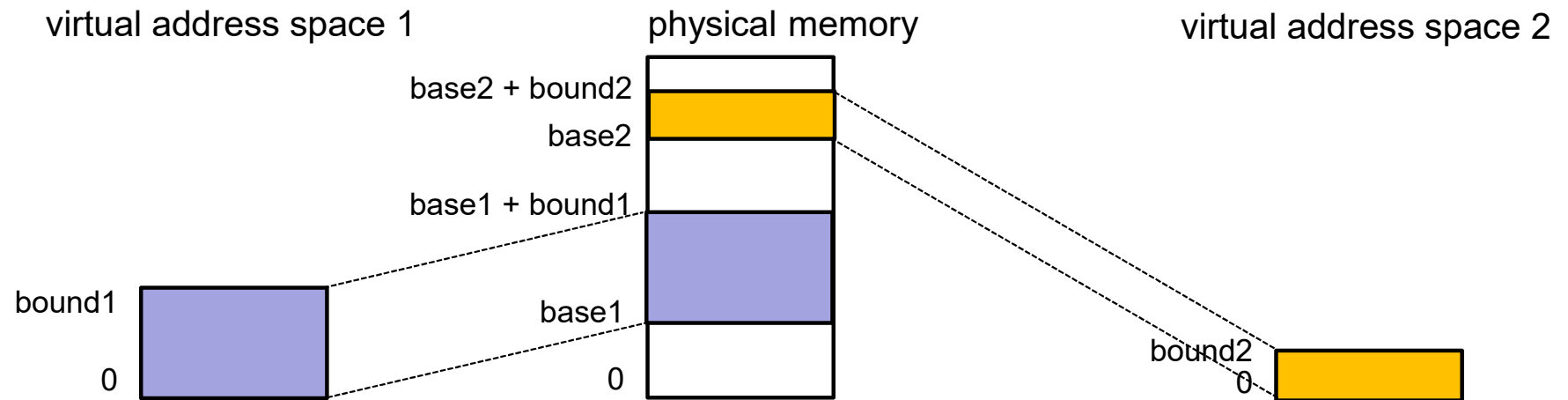
Base and bound: MMU translation algorithm



```
if (virtual address >= bound) {  
    trap to kernel; OS kills process (segmentation violation)  
    or retries access  
} else {  
    physical address = virtual address + base  
}
```

How to switch address spaces?

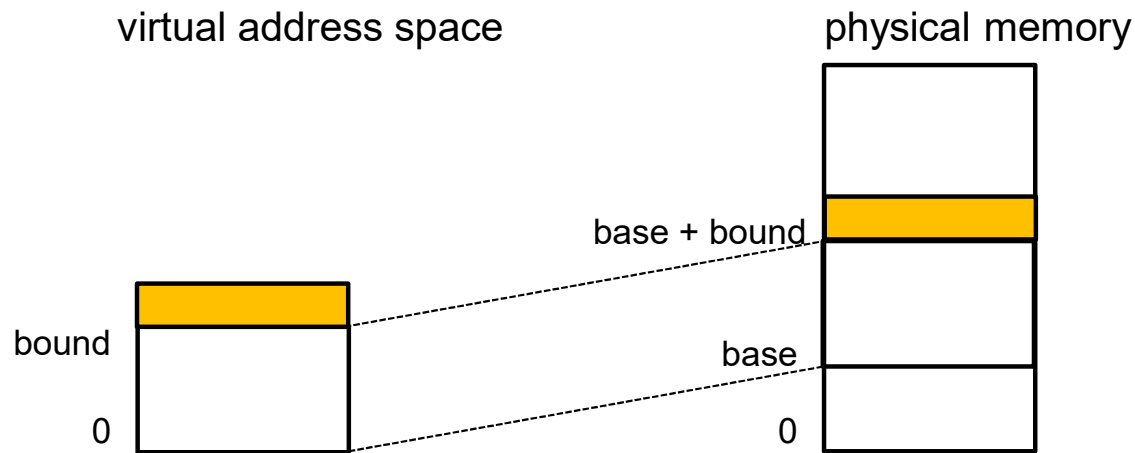
Base and bound: switching address spaces



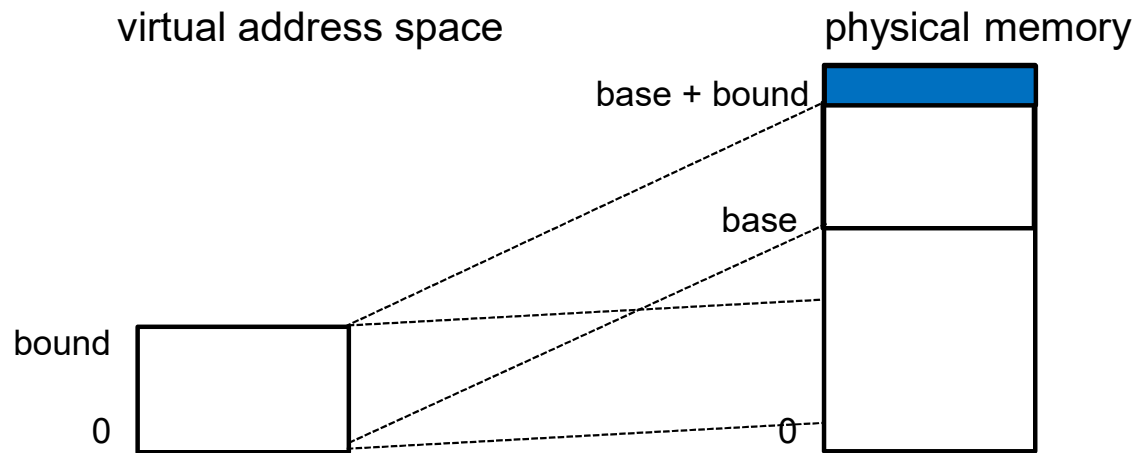
Base and bound

- Fast
- Simple hardware support
- ✓ Address independence?
- ✓ Protection?
- ✗ Large address space?
- Flexibility?

Base and bound: how to grow address space?



Base and bound: how to grow address space?



Is process aware of this move?

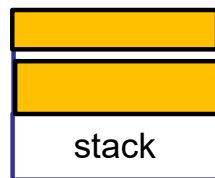
+ Transparent

- Slow

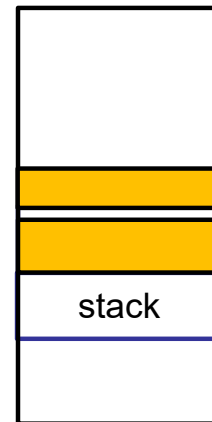
Base and bound: how to grow multiple regions in address space?

- Easy to grow top region (e.g., heap)
- How to grow lower region (e.g., stack)?
 - Problem?

virtual address space



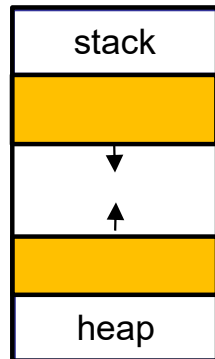
physical memory



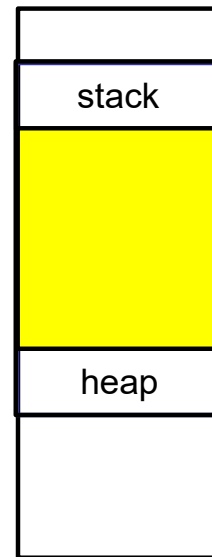
Base and bound: how to grow multiple regions in address space?

- Must reserve space for each region in virtual address space
- **Problem?**

virtual address space

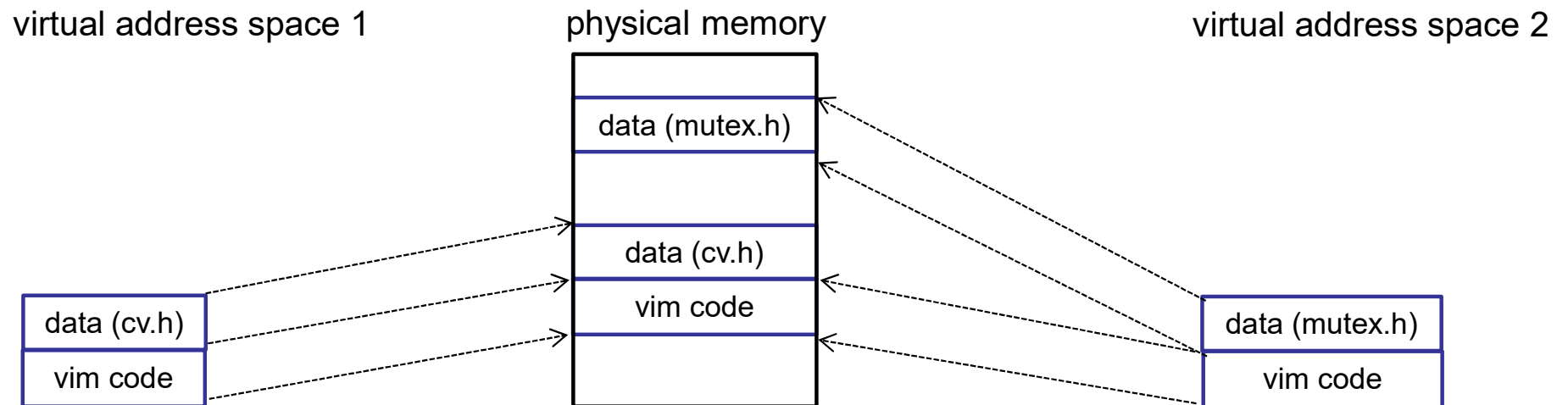


physical memory



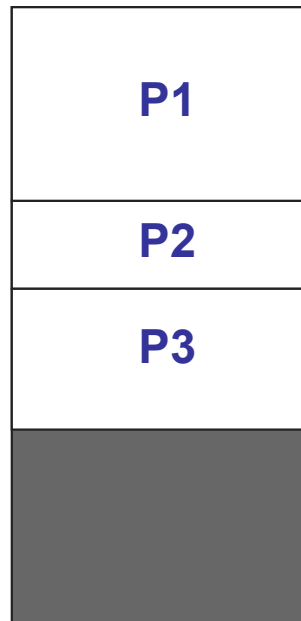
Base and bound: sharing

- Can't share part of an address space between processes



External fragmentation

- Processes come and go, leaving a mishmash of available memory regions
 - Wasted memory between allocated regions



Base and bound

- Pros:
 - Fast
 - Simple hardware support
- Cons:
 - Does not support large address spaces
 - Growing address space may be slow
 - Need to reserve space to allow multiple regions to grow
 - Can't share part of address space
 - External fragmentation
- Root cause: requires **entire address space** to be contiguous in memory

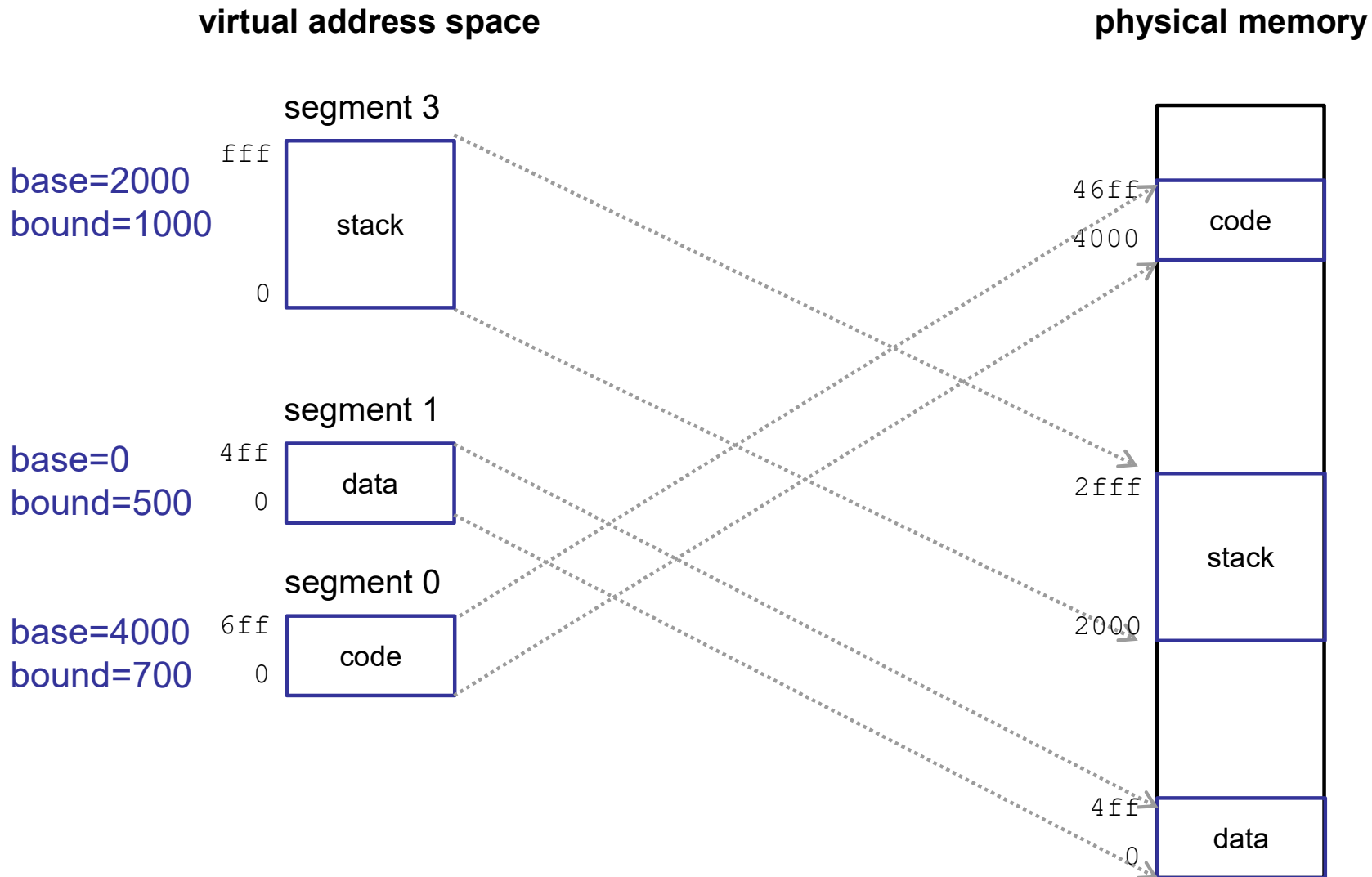
Project 2

- Memory leaks
 - For what type of programs do we care about memory leaks?
 - When should you deallocate memory?
 - What kind of test case shows a memory leak?
 - How to observe a memory leak?

Segmentation

- Divide address space into multiple **segments**
- Segment: region of address space that is:
 - Contiguous in physical memory
 - Contiguous in virtual address space
 - Variable size

Segmentation



Segmentation

Segment #	Base	Bound	Description
3	2000	1000	stack segment
2	n/a	0	unused
1	0	500	data segment
0	4000	700	code segment

- Virtual address: (segment #, offset)
 - Physical address = seg_info[segment #].base + offset
- How to specify the segment #:
 - High bits of address
 - Special register
 - Implicit to instruction opcode

Segmentation: translation

Segment #	Base	Bound	Description
3	2000	1000	stack segment
2	n/a	0	unused
1	0	500	data segment
0	4000	700	code segment

- **Physical address for virtual address (3, 100) ?**
 - $2000 + 100 = 2100$
- **Physical address for virtual address (0, ff) ?**
 - $4000 + ff = 40ff$
- **Physical address for virtual address (1, 2000) ?**
 - invalid (illegal) → segmentation violation/core dump
- **Physical address for virtual address (2, ff) ?**
 - invalid (illegal) → segmentation violation/core dump

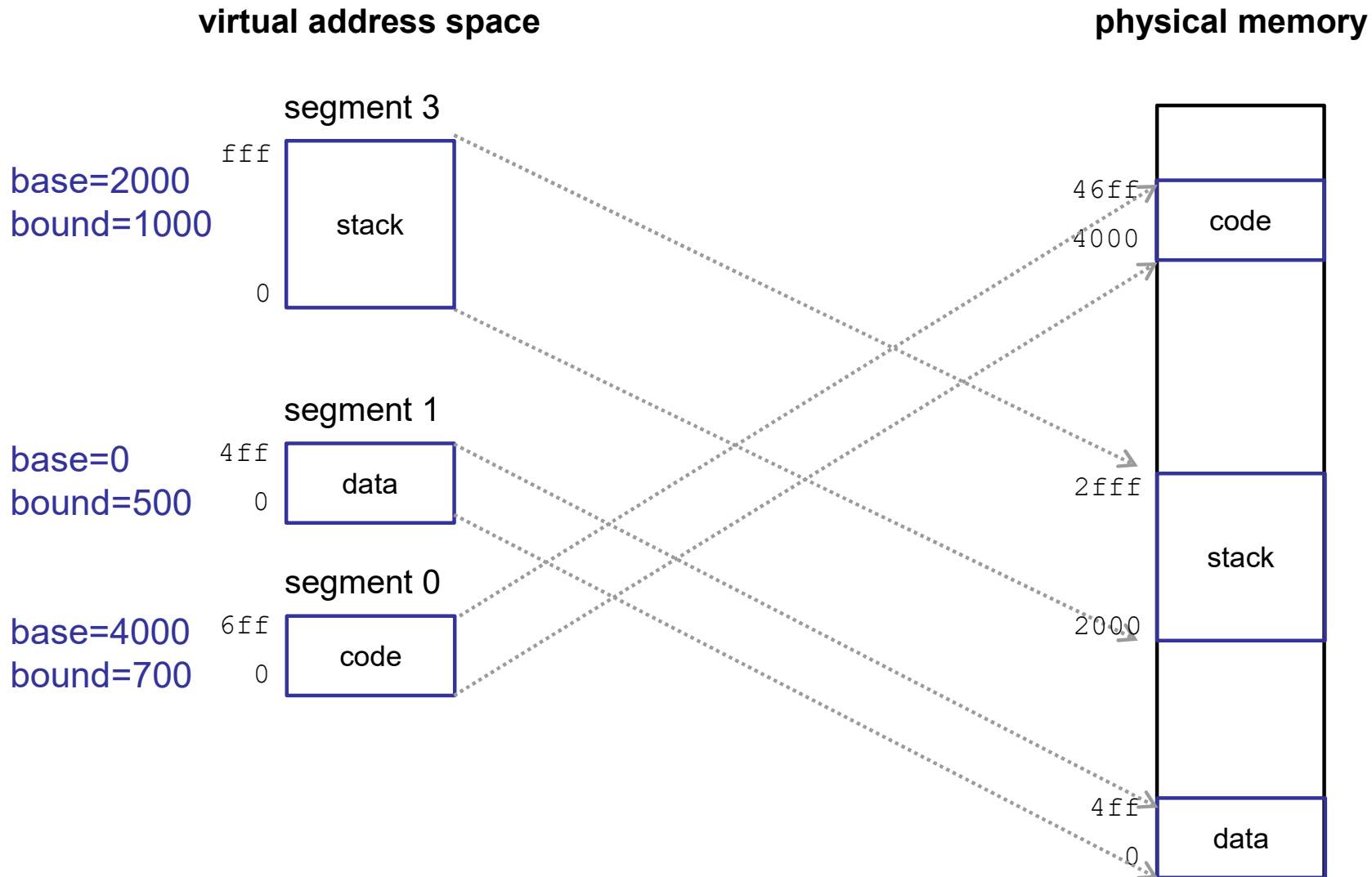
Valid vs. invalid addresses

- Not all virtual addresses are valid
 - Valid → address is part of virtual address space
 - Invalid → virtual address is illegal to access
 - » Accessing invalid address causes trap to OS
- In segmentation, a virtual address can be invalid due to:
 - Out of bound offset
 - Invalid segment number

Segmentation: protection

Segment #	Base	Bound	Description	Protection
3	2000	1000	stack segment	read/write
2	n/a	0	unused	
1	0	500	data segment	read/write
0	4000	700	code segment	read

Segmentation



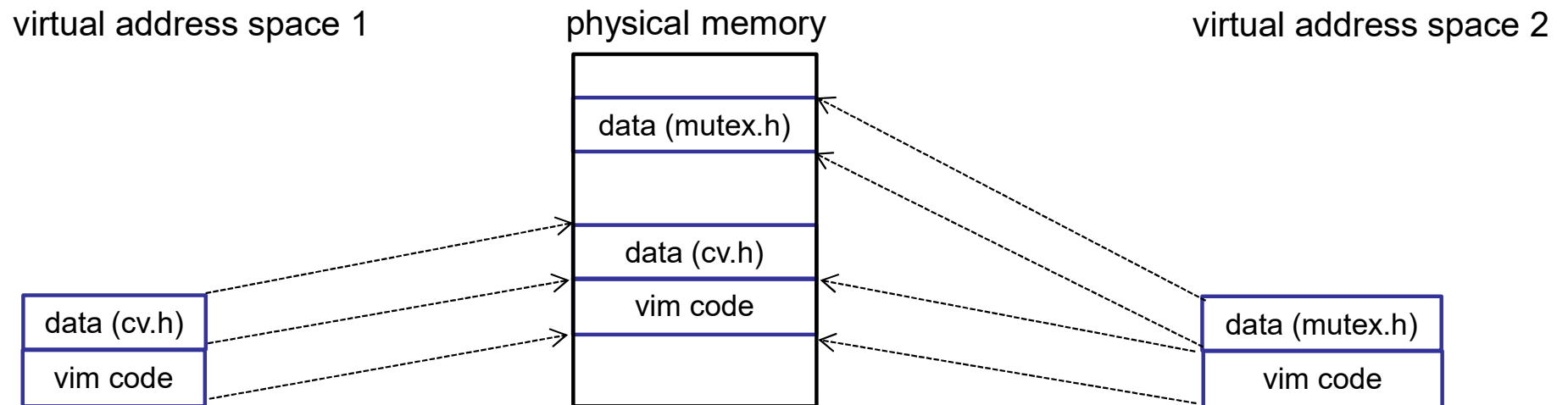
Segmentation: translation algorithm

```
(base, bound, protection) = segment_info[segment]

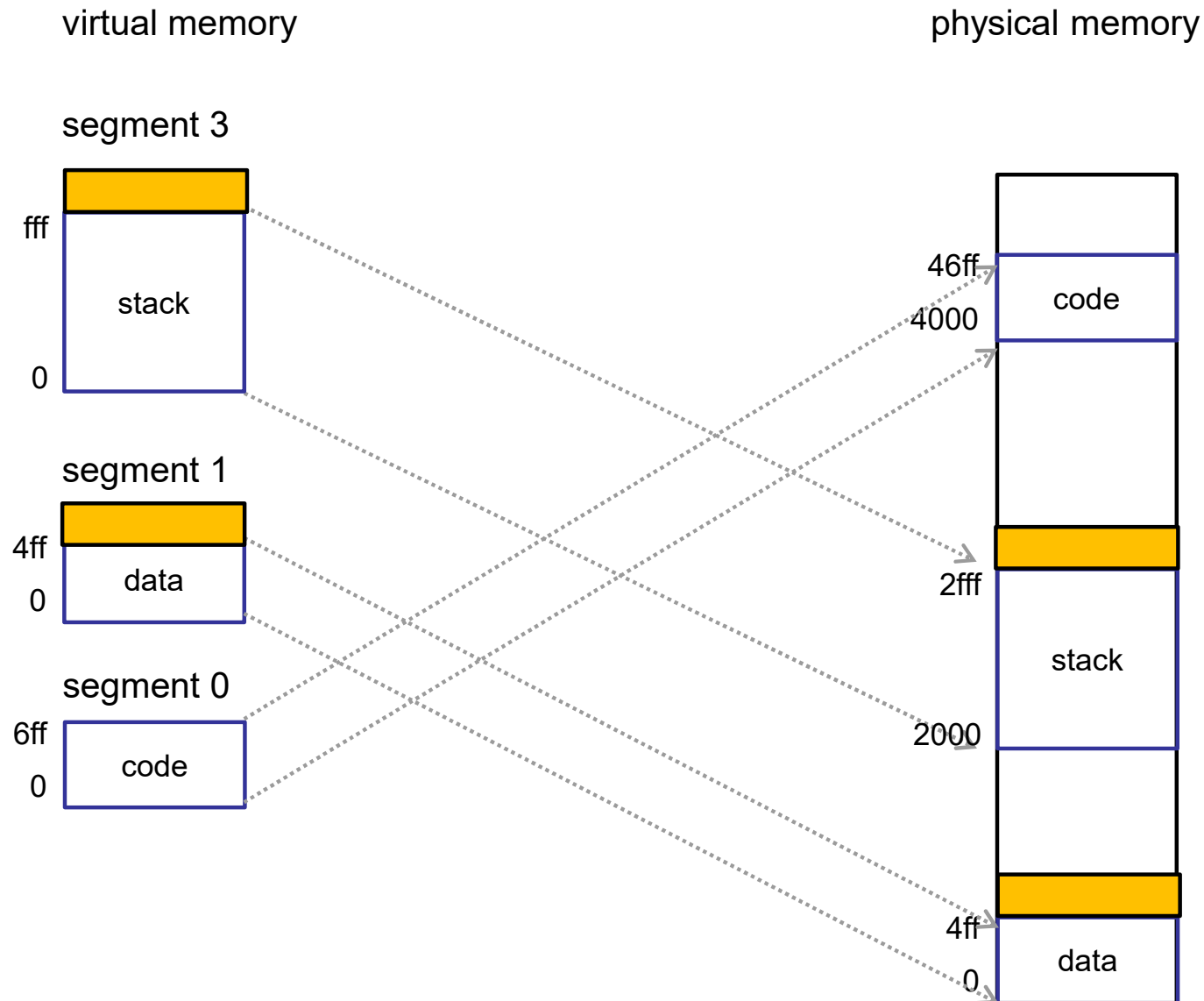
if (segment is invalid or protected, or offset >= bound) {
    trap to kernel
} else {
    physical address = offset + base
}
```

- What must be changed to switch address spaces?

Segmentation supports partial sharing



Segmentation allows multiple growing regions



Segmentation: problems

- Growing address space still slow (may need to copy segment)
- External fragmentation
- Does not support large address spaces?
 - Each segment must fit in memory
- How can we:
 - Make memory allocation easy
 - Not have to worry about external fragmentation
 - Allow address space to be larger than physical memory